

Week 2 Lab Handout

API Planning — From App Features to API Needs

Time: 110 minutes

Bring: laptop, Week 1 audit, ring position

Before you start: planning is not design

Planning answers **why and for whom**. Design (next week) answers **what shape**. Skip straight to endpoints without planning first, and you get an API that serves *your* convenience, not your consumer's actual need — which means redesigning it later anyway, except now with code already built on top of it.

Today you're not writing any code and you're not designing any endpoints. You're figuring out, with evidence instead of guesses, what your ring partner actually needs from you — and what you can actually get from your upstream partner.

Part A — The Consumer-First Mindset (10 min)

Your downstream partner (the team after you in the ring) is your API's **customer**. The question that matters isn't "*what data do I have?*" — it's "*what problem does my partner need solved?*" Those are different questions, and they produce different APIs.

Worked example: Task Tracker → Dashboard

Imagine a **Task Tracker** app exposing data to a **Dashboard** app that displays an upcoming-deadlines widget.

If Task Tracker plans from "*what data do I have,*" they might build full CRUD: create, read, update, delete tasks — because that's what their own app does internally.

But a 10-minute conversation reveals Dashboard only ever needs to **read** tasks — it never creates or edits them. That one conversation just saved Task Tracker from building, documenting, testing, and securing three endpoints nobody was ever going to call.

This is the whole point of planning before design: the conversation is cheap. The unused endpoint is not.

Part B — Interview Your Downstream Partner (15 min)

Sit down with the team that will consume *your* API. Ten minutes, structured — don't let it turn into a general chat. Ask:

1. "What data or actions from our app would actually be useful to you?"
2. "Would you ever need to **create or change** data in our system, or only **read** it?"
3. "How often would you need this — once per page load? Real-time? Once a day?"
4. "Is there anything about our app you assumed you could access that actually isn't there?"

Take notes. You'll need specifics (not impressions) to write real needs statements in Part D.

Part C — Interview Your Upstream Partner (15 min)

Now go the other direction: sit down with the team whose API *you* will consume, and find out what's actually available to you. Ask:

1. "What resources or data does your app manage that we might use?"

2. "What can we actually do with it — just read, or also create/update/delete?"
3. "Is there anything sensitive or restricted we shouldn't expect access to?"
4. "How up-to-date is the data likely to be when we fetch it?"

This conversation prevents you from planning your Week 3+ work around something that was never going to be available in the first place.

Part D — Write Your API Needs Statements (40 min)

The format

Every needs statement follows this exact shape:

"[Consumer team] needs to [verb] [resource] in order to [reason]."

Multiple statements are expected — most APIs serve more than one need. Write **3 to 6** statements, one per real need you uncovered in Part B.

Vague vs. specific — this is what gets graded

Vague (avoid this)	Specific (aim for this)
"Dashboard needs to get task data in order to work."	"Dashboard needs to read a list of active tasks with due dates, in order to display an upcoming-deadlines widget."
"Library App needs access to student info in order to function."	"Library App needs to read a student's active loan count in order to block new checkouts once the limit is reached."

The vague version could describe almost any API. The specific version tells you exactly what data, what shape, and what it's for — which is precisely what you need to actually design an endpoint next week. If your statement could apply to five different features, it's not specific enough yet.

Don't forget the non-functional details

For each statement, jot down (briefly — this isn't a deep dive yet, just seeding what's coming in later weeks):

- **Freshness:** does this need real-time data, or is "updated within the hour" fine?
- **Volume:** roughly how often will this get called — once per page load, or constantly?
- **Auth:** is this data sensitive enough to require authentication, even at a glance?

You'll revisit all three properly in Weeks 4 and 9 — for now, a one-line guess per statement is enough.

Part E — Sanity-Check Against Your Week 1 Audit (15 min)

Pull up your Week 1 resource/action audit. For **every** needs statement you just wrote, confirm it maps back to something on that list.

- If a need doesn't map to anything in your audit, you either missed something in Week 1, or your partner is asking for something your app genuinely doesn't do yet. **Flag it** — write one line noting the gap, and raise it with your instructor before moving on.
- If something in your Week 1 audit maps to *no* needs statement at all, that's fine — not everything your app does needs to become part of the API. Just don't be surprised if it never shows up again.

This step exists to catch wishful thinking before it becomes a Week 3 endpoint list built on a need that was never real.

Part F — Write Your Reflection and Finalize (15 min)

Add one paragraph to the bottom of your document reflecting on **what surprised you** in the partner interviews. Genuine engagement here means something specific — “nothing surprised us” is a signal you didn’t ask hard enough questions in Part B/C, not a real reflection.

Student deliverable

Create `API_NEEDS.md` at your repo root, containing:

1. **3–6 needs statements**, each in the exact format from Part D
2. **One reflection paragraph** on what surprised you in the partner interview

Commit and push it:

```
git add API_NEEDS.md
git commit -m "Week 2: API needs statements"
git push
```

(Still no branch-naming convention to follow — that starts once you're working out of the template repository your instructor hands you.)

This week's deliverable checklist

- ☐ Downstream partner interviewed, notes taken
- ☐ Upstream partner interviewed, notes taken
- ☐ 3–6 needs statements written in the required format
- ☐ Every statement checked against the Week 1 audit; any gaps flagged
- ☐ Reflection paragraph written and genuine
- ☐ `API_NEEDS.md` committed and pushed

Rubric (20 pts)

Criterion	Points
Needs statements follow the required format	6
Statements reflect actual partner interview (not guessed)	6
Needs are specific and scoped (not vague, e.g. “give data”)	5
Reflection paragraph shows genuine engagement	3

Common problems and quick fixes

Problem	Fix
Your partner team isn't ready / hasn't shown up	Start with Part D using your best guess from your Week 1 audit, then update your statements the moment you do connect — don't let a scheduling gap block your whole session

Every need you write sounds like “give data”	Go back to Part B’s notes and find the actual UI element or workflow your partner described — a needs statement should be specific enough that you could sketch the screen it supports
All your needs are read-only and that feels too easy	That’s a legitimate outcome, not a failure — re-read the Task Tracker/Dashboard example above. A short list of real needs beats a padded list of imagined ones
A need doesn’t map to anything in your Week 1 audit	Flag it per Part E rather than quietly dropping or forcing it — this is exactly the kind of gap your instructor wants to hear about before Week 3
You’re not sure if something counts as a “non-functional” detail	Freshness, volume, and auth are the only three to note this week — if it’s not one of those three, leave it for later weeks

Next week: turning these needs statements into a real endpoint list — REST conventions, resource modeling, and your first peer review.